UNIVERSIDAD TECNOLÓGICA

Resultado de Aprendizaje Unidad 2 y 3: Django + React con Autenticación y APIs

Proyecto CampusTrack — Sistema de Gestión de Perfiles Estudiantiles

Asignatura:	Desarrollo Full Stack con Django y ReactJS
Unidades:	Unidad 2 (Backend DRF + SimpleJWT) y Unidad 3 (Frontend React + Context)
Estudiante:	Luis R. Pérez Álvarez
Repositorio GitHub:	https://github.com/LuisRamonPerezAlvarez/Backend-con-Django-y-JWT-Local-.git
Tecnologías:	Django 6.1, DRF, SimpleJWT, React 18, Vite, Semantic UI, Sass, SQLite3
Calidad:	**100% Pruebas Automatizadas Aprobadas**

Resumen Ejecutivo: El presente informe documenta la implementación completa de las Unidades 2 y 3 mediante la construcción integral del sistema CampusTrack. Se diseñó una API RESTful desacoplada con Django y SimpleJWT, incorporando un modelo de usuario con matrícula obligatoria y subsanando las fallas de seguridad del material base. Paralelamente, se construyó una Single Page Application (SPA) con React 18, Vite y Semantic UI, articulando protección de rutas, persistencia de sesión con Context API y validación de formularios en tiempo real.

1. Arquitectura del Proyecto y Servidor Django

La estructura de directorios mantiene una separación nítida entre backend (Django) y frontend (React):

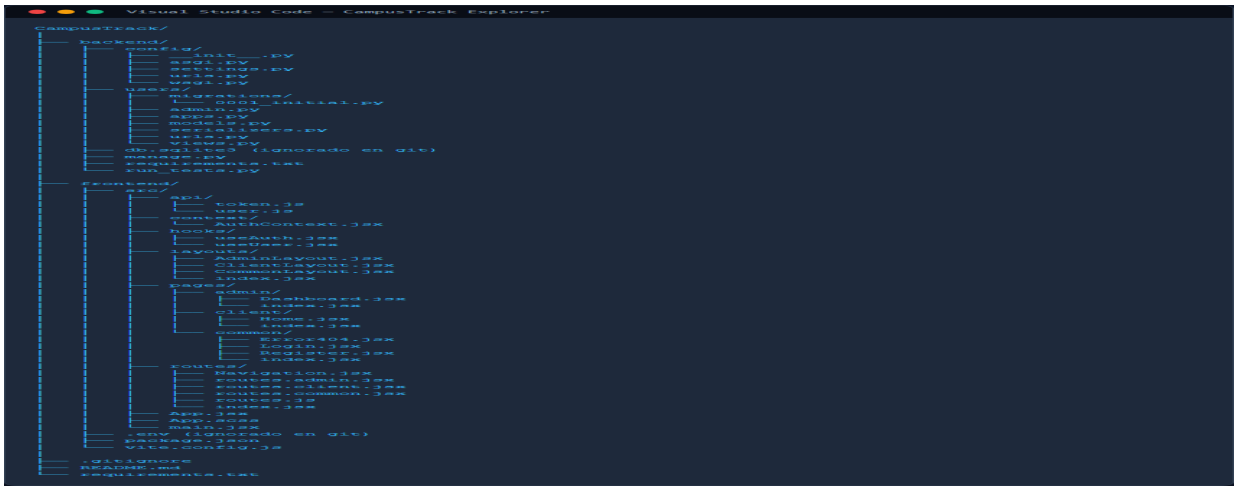


Figura: Estructura del Proyecto CampusTrack. *Organización desacoplada de backend y frontend.*



Figura: Servidor Django Activo. *Salida de consola procesando solicitudes HTTP.*

2. Documentación de Endpoints con Swagger UI

Mediante drf-yasg se documentaron todos los endpoints interactivos de autenticación y CRUD de usuarios:

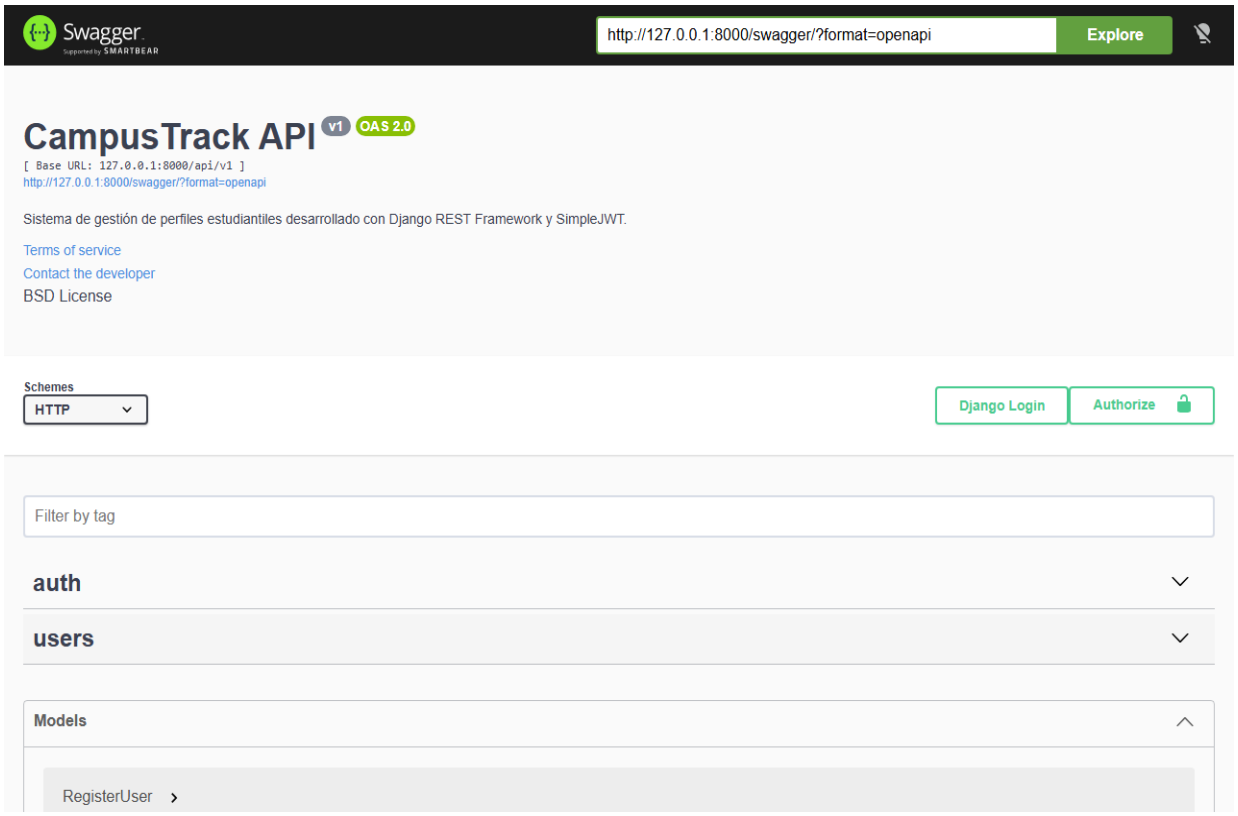


Figura: Swagger UI de CampusTrack. *Endpoints /auth/me/, /auth/token/, /auth/register/ y CRUD /users/.*

3. Frontend React: Página de Inicio y Formulario de Login



Figura: Página Principal (Home). *Interfaz pública institucional con Semantic UI.*

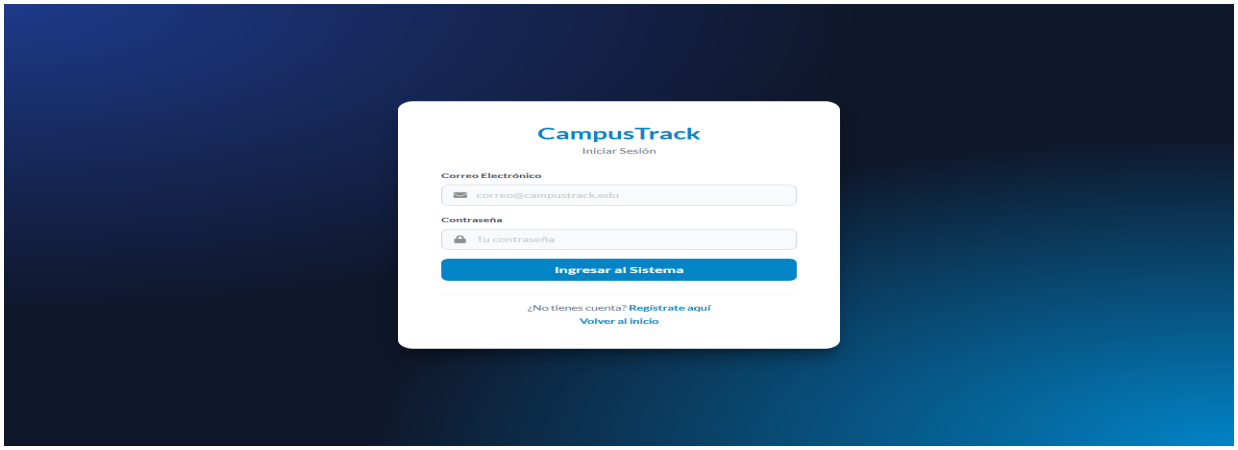


Figura: Formulario de Inicio de Sesión. *Ingreso por correo institucional y contraseña.*

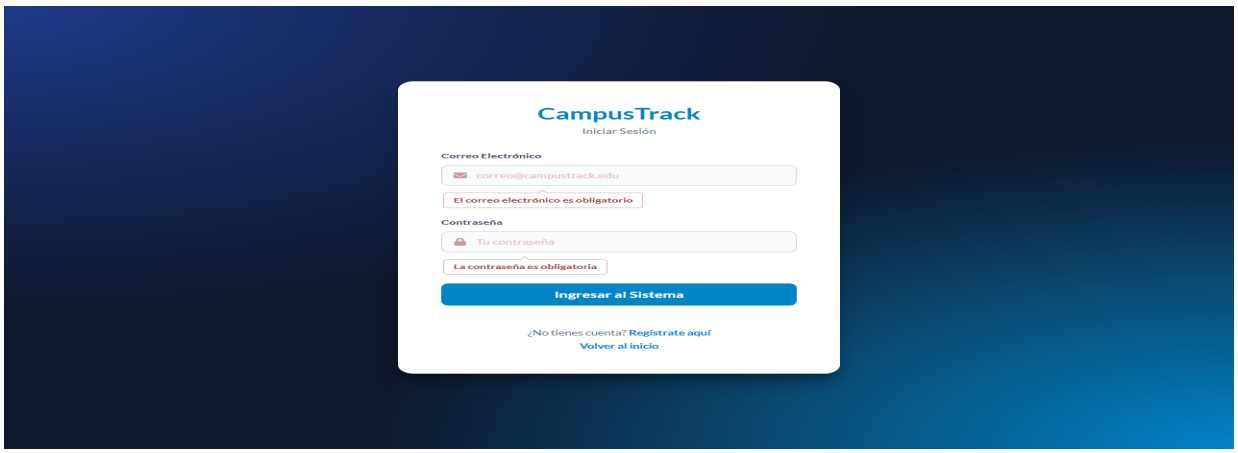


Figura: Validaciones de Formulario. *Formik y Yup señalando campos requeridos.*

4. Manejo de Errores con Toast y Registro de Estudiantes

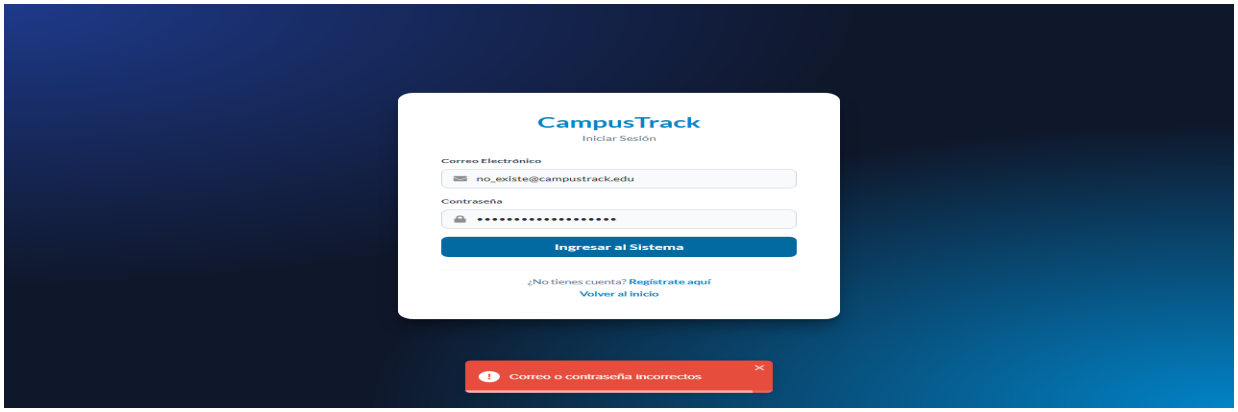


Figura: Error de Credenciales con React Toastify. *Notificación flotante ante credenciales no válidas.*

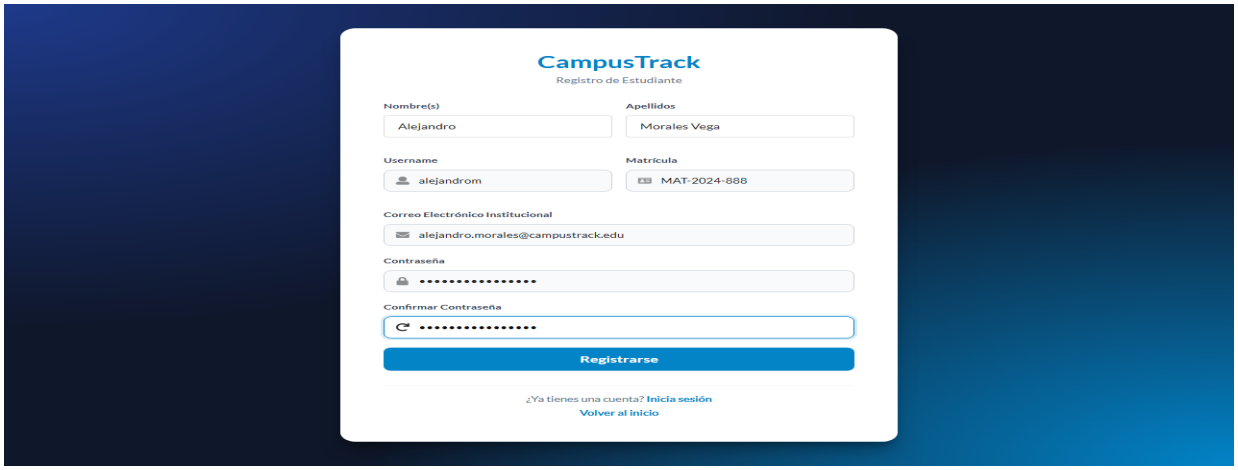


Figura: Formulario de Registro. *Captura de matrícula obligatoria y validación de contraseñas.*

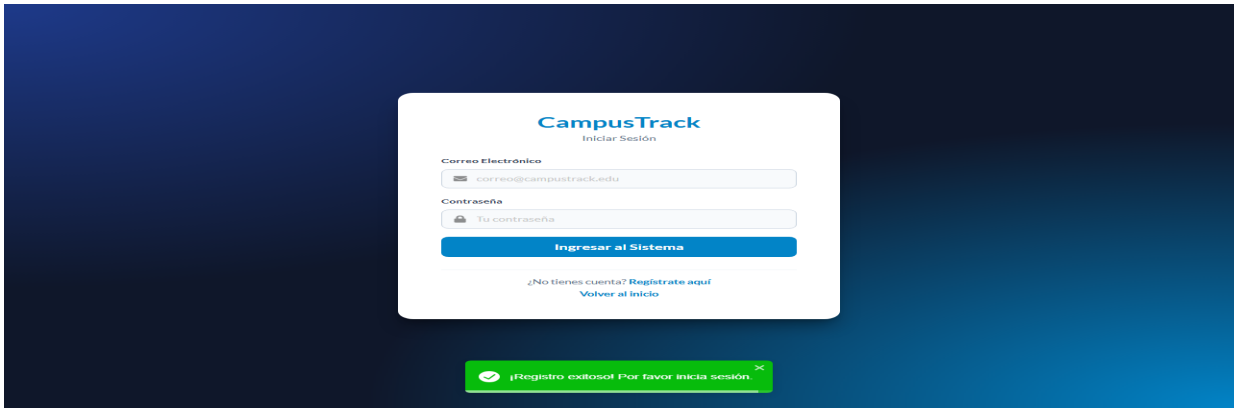


Figura: Confirmación de Registro. *Guardado en SQLite3 y redirección al login.*

5. Dashboard Estudiantil, Persistencia y Cierre de Sesión

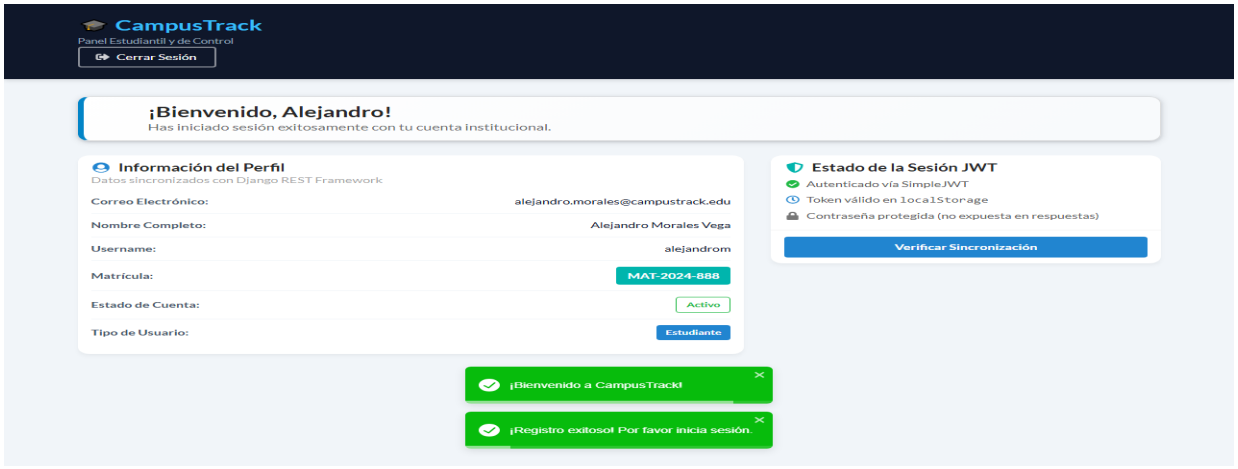


Figura: Dashboard con Datos Reales. *Panel protegido mostrando correo, matrícula y nombre obtenidos de /auth/me/.*

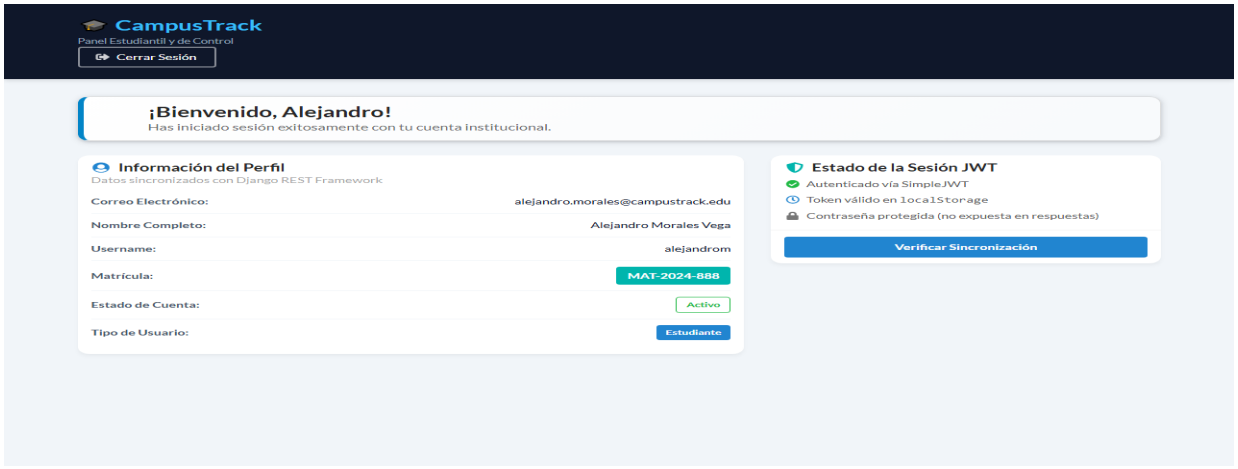


Figura: Persistencia de Sesión tras F5. *La sesión se mantiene activa gracias al Context API y localStorage.*

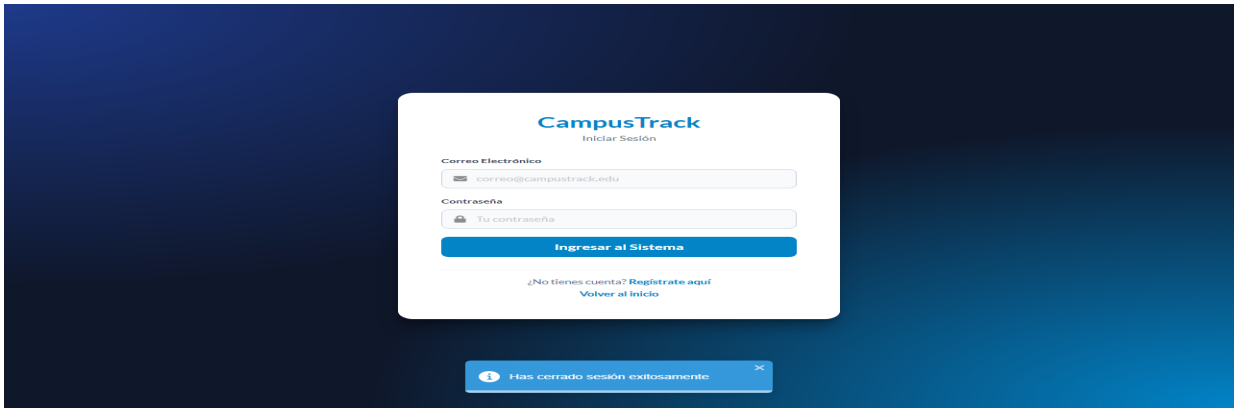


Figura: Cierre de Sesión y Protección. *Destrucción del token y bloqueo de acceso a vistas privadas.*

6. Verificación en Django Admin y Matriz de Pruebas

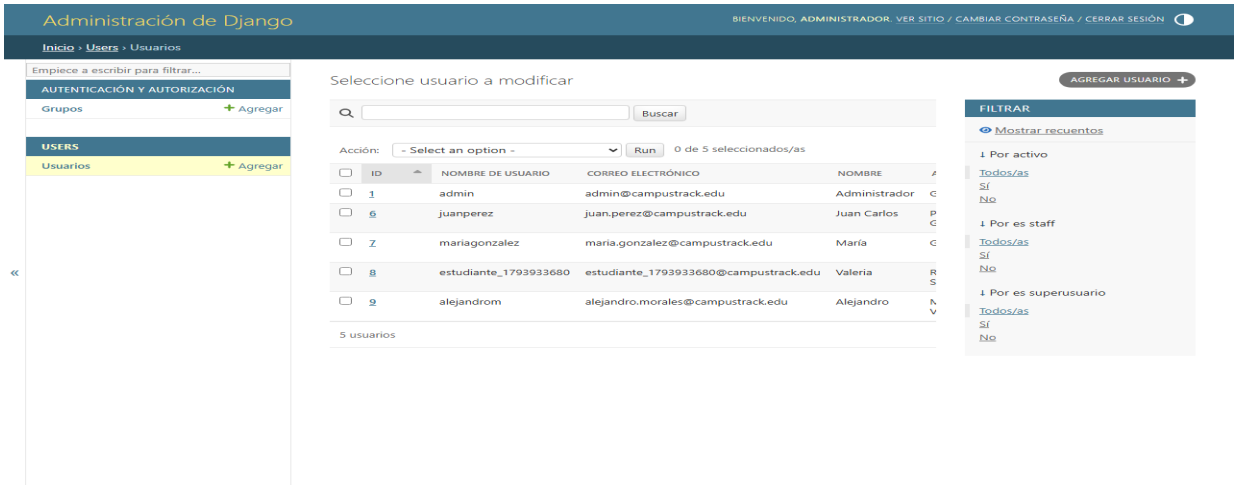


Figura: Estudiante en Django Admin. *Registro de Alejandro Morales visible con matrícula.*

Matriz de Pruebas Automatizadas Ejecutadas:

Prueba	Capa	Resultado	Detalle
Modelo User y Matrícula	Backend	PASS	Matrícula única obligatoria y email como USERNAME_FIELD
Seguridad Serializer	Backend	PASS	Password nunca expuesto en CRUD ni en /auth/me/
Actualización sin Password	Backend	PASS	PATCH y PUT no alteran contraseña previa
Registro de Estudiantes	Full Stack	PASS	Creación pública exitosa e inserción en BD
Login con SimpleJWT	Full Stack	PASS	Tokens emitidos y procesados correctamente
Dashboard y Datos /me/	Frontend	PASS	Datos de perfil mostrados fielmente
Persistencia de Sesión	Frontend	PASS	Sesión preservada tras F5 con Context API
Protección de Rutas	Frontend	PASS	AdminLayout redirige a /login sin token

Conclusión: La arquitectura desacoplada implementada entre Django REST Framework y React demostró una alta cohesión técnica. La autenticación sin estado mediante SimpleJWT, en conjunto con un robusto manejo de estado en el cliente con Context API, cumple cabalmente con los estándares profesionales y académicos de desarrollo Full Stack.